SPRINT 03 · ANGULAR · LAB04

Observables y Signals

RxJS, subjects, operadores, Signals e interoperabilidad entre ambos

Ejercicios de entrenamiento. Hay tres niveles y cada uno es un proyecto distinto e independiente, así que no necesitas el anterior para empezar. Los enunciados no incluyen la solución: usa «Cómo saber que está bien» para comprobar tu trabajo.

Nivel	Proyecto	Practicas	Tiempo orientativo
Fácil	Marcador de baloncesto	`signal`, `computed`, `effect`, tipos de Subject y operadores básicos	1 h
Medio	Buscador de películas	`debounceTime`, `distinctUntilChanged`, `switchMap`, `catchError` y `toSignal`	2-2,5 h
Difícil	Panel de vuelos	Actualización periódica, flujos compartidos, `exhaustMap`, `toObservable` y `takeUntilDestroyed`	4-5 h

Trabaja con Angular 22, componentes standalone, `@if` / `@for` e `inject()`. Crea cada proyecto por separado con `ng new <nombre> --style=scss --ssr=false`. Si prefieres decoradores (`@Input`, `@Output`) o funciones (`input()`, `output()`), ambas opciones valen. Angular 22 no usa Zone.js: guarda en un `signal` el estado que cambie fuera de un evento de plantilla (temporizadores, HTTP o suscripciones al router), o la vista no se actualizará.

Fácil

1 h

`marcador-baloncesto`

Marcador de baloncesto

Practicas: `signal`, `computed`, `effect`, tipos de Subject y operadores básicos

Primero vas a construir un marcador con Signals. Después razonarás qué emite cada tipo de Subject con la misma secuencia de puntos.

Parte A: el marcador

1. Crea los signals `local` y `visitante`, ambos a 0, y un signal `historial` de tipo `string[]`.
2. Añade botones de +1, +2 y +3 para cada equipo. Actualiza con `update`.
3. Crea con `computed`: `diferencia` (valor absoluto) y `lider` («Local», «Visitante» o «Empate»).
4. Cada canasta añade una línea al historial **sin mutar el array**.
5. Crea un `effect` que registre el marcador en consola, y un botón «Reiniciar».

Parte B: ¿qué sale por consola?

Sin ejecutar, escribe la salida con cada una de las tres definiciones de `s` y con la secuencia común. Comprueba después tu respuesta.

```
// 1) const s = new Subject<number>();
// 2) const s = new BehaviorSubject<number>(0);
// 3) const s = new ReplaySubject<number>(1);

s.next(2);
s.subscribe(v => console.log('A', v));
s.next(5);
s.subscribe(v => console.log('B', v));
s.next(7);
```

Y este flujo de operadores

```
of(2, 3, 1, 3, 2, 2).pipe(
  filter(puntos => puntos > 1),
  map(puntos => puntos * 10),
  take(3)
).subscribe({
  next: v => console.log(v),
  complete: () => console.log('fin'),
});
```

Cómo saber que está bien

- Sumar 3 al local y 1 al visitante da diferencia 2 y líder «Local».
- El `effect` se ejecuta al cargar y cada vez que cambia un marcador.

Preguntas

- ¿Por qué `historial().push(linea)` no actualizaría la vista?
- ¿Se ejecuta el `computed` si nadie lee su valor?

Medio

2-2,5 h

`buscador-peliculas`

Buscador de películas

Practicas: `debounceTime`, `distinctUntilChanged`, `switchMap`, `catchError` y `toSignal`

Un buscador debe evitar peticiones innecesarias y no mostrar resultados antiguos. El servicio de abajo responde con retrasos variables para provocar condiciones de carrera.

Servicio simulado

```
@Injectable({ providedIn: 'root' })
export class PeliculasApi {
  private datos = ['Alien', 'Amelie', 'Avatar', 'Casablanca', 'Coco', 'Matrix', 'Memento', 'Up'];

  buscar(texto: string): Observable<string[]> {
    if (texto === 'error') {
      return timer(300).pipe(switchMap(() => throwError(() => new Error('Fallo del servidor'))));
    }
    const retraso = 200 + Math.random() * 1500;
    const hallados = this.datos.filter(p => p.toLowerCase().includes(texto));
    return of(hallados).pipe(delay(retraso));
  }
}
```

Requisitos

1. Usa un `FormControl` para el texto y parte de su `valueChanges`.
2. Encadena: normalizar con `trim` y minúsculas, `debounceTime(300)`, `distinctUntilChanged()` y `switchMap` hacia la API.
3. No busques con menos de 2 caracteres: en ese caso, muestra la lista vacía sin llamar a la API.
4. Modela el resultado como un único estado: `{ estado: 'inactivo' | 'cargando' | 'listo' | 'error'; peliculas: string[] }`. Emite «cargando» con `startWith` dentro del `switchMap`.
5. Controla el error con `catchError` **dentro** del `switchMap`, de modo que se pueda seguir buscando después.
6. Convierte el flujo en signal con `toSignal`. La plantilla no usa `subscribe` ni el pipe `async`.
7. Sustituye temporalmente `switchMap` por `mergeMap`, escribe deprisa «ma», «mat», «matr» y anota qué sucede.

Cómo saber que está bien

- Escribir rápido provoca una sola petición, no una por tecla (compruébalo con `tap`).
- Tras buscar «error» y ver el mensaje, buscar «coco» funciona.
- Con `switchMap` nunca se muestran resultados de una búsqueda anterior.

Preguntas

- Mueve `catchError` fuera del `switchMap`. ¿Por qué deja de funcionar el buscador?
- Explica en una frase cuándo usarías `switchMap`, `mergeMap`, `concatMap` y `exhaustMap`.
- ¿Por qué aquí conviene `initialValue` en `toSignal`?

Difícil

4-5 h

`panel-vuelos`

Panel de vuelos

Practicas: Actualización periódica, flujos compartidos, `exhaustMap`, `toObservable` y `takeUntilDestroyed`

Un panel de aeropuerto se actualiza solo cada 5 segundos. Combina RxJS para los flujos de tiempo con Signals para el estado que se muestra en pantalla.

API simulada

```
export type EstadoVuelo = 'en hora' | 'retrasado' | 'embarcando' | 'cancelado';

export interface Vuelo {
  codigo: string;
  destino: string;
  puerta: string;
  estado: EstadoVuelo;
  retrasoMin: number;
}

// Implementa getVuelos(): Observable<Vuelo[]>:
// 1) devuelve 10 vuelos inventados con delay(400);
// 2) en cada llamada, cambia al azar el estado de 1 o 2 vuelos;
// 3) falla (throwError) en una de cada 6 llamadas.
```

Requisitos

1. Crea `VuelosStore` (`providedIn: 'root'`). Actualiza con `timer(0, 5000)` y `exhaustMap`, para no acumular peticiones si una tarda.
2. Comparte el flujo con `shareReplay({ bufferSize: 1, refCount: true })`. Dos componentes suscritos deben causar una sola petición por ciclo.
3. Un error no detiene el panel: conserva los últimos datos, muestra un aviso con la hora del último dato correcto y sigue actualizando.
4. Añade «Pausar» y «Reanudar» con un signal `pausado` convertido con `toObservable` y `switchMap` hacia `EMPTY` o la actualización.
5. Añade «Actualizar ahora», sin esperar al siguiente ciclo y sin peticiones solapadas.
6. Filtros de destino y estado en signals. Crea un `computed` `visibles` y otro `resumen` (`total`, `retrasados`, `retrasoMedio`).
7. Cuando un vuelo pase a «cancelado», publica una alerta. Compara el valor nuevo con el anterior (`pairwise` es una buena opción).
8. Guarda los vuelos seguidos en `localStorage` con un `effect`. Cancela todas las suscripciones de componente con `takeUntilDestroyed`.

Cómo saber que está bien

- La consola muestra una sola petición cada 5 segundos, aunque haya varios componentes.
- Al pausar no hay peticiones; al reanudar, se pide de inmediato.
- Al salir del panel (`refCount` a 0), las peticiones se detienen.
- Una alerta de cancelación aparece una sola vez por vuelo.

Preguntas

- Sustituye `exhaustMap` por `switchMap`. ¿Qué pasaría si la API tardara más de 5 segundos en cada respuesta?
- ¿Qué ocurre con `refCount: false` al salir del panel?
- ¿Qué partes de este ejercicio son más naturales con RxJS y cuáles con Signals? Justifícalo.